39. Cloud VPCs: Subnets, Route Tables, Gateways

39. Cloud VPCs: Subnets, Route Tables, Gateways

Phase: Phase 12 — Cloud, containers, and modern nets
Estimated time: 2 days
Slug: `39-cloud-vpc`

Learning objectives

- Map VPC/subnet/route table/IGW/NAT
- Design public vs private subnets
- Avoid overlapping CIDRs for peering
- Read cloud route tables like `ip route`

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Private subnet instances cannot apt-get. Route tables, IGW vs NAT gateway, and subnet roles in a VPC explain egress.

Core concepts

What you must be able to explain:

- Map VPC/subnet/route table/IGW/NAT
- Design public vs private subnets
- Avoid overlapping CIDRs for peering
- Read cloud route tables like `ip route`

Mental model for `39-cloud-vpc` (exercise kind `vpc_cidrs`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
docker network ls 2>/dev/null; ip -br addr | head
# kubectl get svc,ep -A 2>/dev/null | head
```

Sample output:

```
NETWORK ID ... bridge ...
eth0 ... 10.0.0.5/24
cni0 ... 10.244.0.1/24
```

Interpretation: Bridge/CNI interfaces explain why `localhost` inside a container is not the host — classic false-positive health check.

Example 2

```
getent hosts kubernetes.default.svc.cluster.local 2>/dev/null || echo 'CoreDNS
path: svc.namespace.svc.cluster.local'
```

Sample output:

```
10.96.0.1 kubernetes.default.svc.cluster.local
```

Interpretation: Cluster DNS (CoreDNS) answers service names. Wrong search domain → NXDOMAIN that works with FQDN.

Example 3

```
# Cloud mental model (AWS-style)
echo 'public subnet → IGW; private subnet egress → NAT gateway; SG stateful; NACL
stateless'
```

Sample output:

```
private_subnet_egress_via=NAT-gateway
```

Interpretation: Route tables decide egress path. Security groups dropping return traffic look like timeouts.

Mini-lab

1. Re-type every worked command for **Cloud VPCs: Subnets, Route Tables, Gateways** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
All subnets public	See common mistakes	Re-run lesson commands
Overlapping 10.0.0.0/8 everywhere	See common mistakes	Re-run lesson commands
No egress strategy for private subnets	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Cloud VPCs: Subnets, Route Tables, Gateways

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- All subnets public
- Overlapping 10.0.0.0/8 everywhere
- No egress strategy for private subnets

Exercises

Work in `exercises/39-cloud-vpc/`. Read that folder's `README.md` before coding.

```
cd exercises/39-cloud-vpc
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `private_subnet_egress_via`, `public_subnet_egress_via`, `cloud_route_object`. Each function is a tiny retrieval drill for **Cloud VPCs: Subnets, Route Tables, Gateways** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/39-cloud-vpc/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. Private subnet egress via?
2. Why non-overlapping CIDRs?
3. SG stateful meaning roughly?

Further reading

- https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html